DATA STRUCTURES & ALGORITHMS PROJECT

Sian Hayes (x23339837)

Academic Integrity Number: EPG2425038

GitHub Repo Link:

https://github.com/SianEAH/Data-Structures-and-Algorithms-Project

Theme Chosen:

Mobility Innovation

Description of My Application

My application SmartPark is a car parking application. It is designed to manage car parking in the Docklands area, to innovate how mobility is handled in this area. This application allows users to check the availability of car parking spaces in the Docklands area, as well as reserving those spaces, with some users having a higher priority based on their needs (such as a disability). The application also manages vehicles waiting for car parking spaces when none are currently available. The system also records recent parking activity, such as the most recent vehicle entries and exits. My application takes in user information using GUI forms. It takes in details such as owner name, license plate number and car type for adding a vehicle and license plate number for removing a vehicle, this is handled by a Singley Linked List. The Parking Waiting List and Disabled Waiting List is handled by a Queue and Priority Queue, respectively. Viewing recent parking history is handled by a Stack.

The Problem My Application Solves

Dublin City Centre, and the Docklands in particular, often suffers from heavy congestion due to a range of factors, such as traffic collisions and ongoing building work. It can be difficult to locate car parking spaces because of it. Drivers can often spend a great deal of time driving around looking for a space, adding time to their commute, the congestion in the city, the wastage of fuel and increasing the C02 emissions, which impacts the environment and the lifestyle of those in the city.

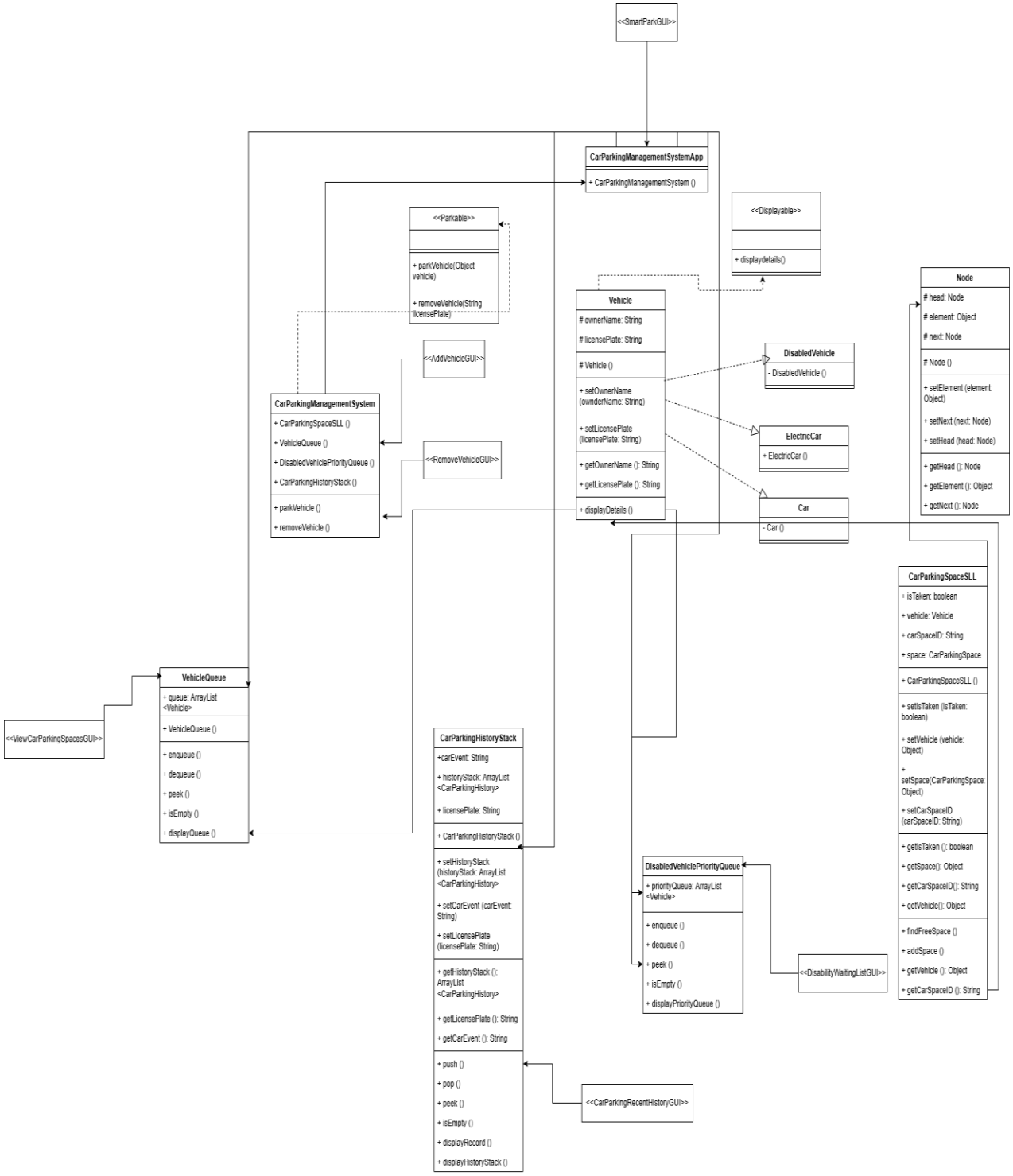
SmartPark can help this issue, as it can track freely available spaces as well as occupied ones. If there are spaces that are free, vehicles are assigned based on priority to these spaces and if no spaces are available at that present time, vehicles will be placed in a queue until a space becomes available. My application allows spaces to be digitally tracked and managed, this increases efficiency and shows how technology can help in the innovation of mobility in the Docklands area.

How it Fits with the Theme

My SmartPark application fits this theme by using emerging digital technologies to help innovate the mobility in the Docklands area, while keeping a people-centred approach.

The availability of parking has a direct impact on traffic congestion, as well as the lifestyle of people in the Docklands area. By digitally managing car parking in the Docklands, it will help to improve congestion and ensure people are not needlessly spending more time on the roads than they must. By tracking parking availability and managing vehicles through a priority system and a waiting queue, my application improves efficiency and reduces unnecessary traffic on the already busy roads. This demonstrates how digital solutions can contribute to smarter and more efficient urban mobility in the Docklands, to try and meet our goal of a “Smarter Docklands.”

Class Diagram


<<SmartParkGUI>>
CarParkingManagementSystemApp
+ CarParkingManagementSystem ()
<<Parkable>>
+ parkVehicle(Object vehicle)
+ removeVehicle(String licensePlate)
<<Displayable>>
+ displaydetails()
Node
head: Node
element: Object
next: Node
Node ()
+ setElement (element: Object)
+ setNext (next: Node)
+ setHead (head: Node)
+ getHead (): Node
+ getElement (): Object
+ getNext (): Node
Vehicle
ownerName: String
licensePlate: String
Vehicle ()
+ setOwnerName (ownderName: String)
+ setLicensePlate (licensePlate: String)
+ getOwnerName (): String
+ getLicensePlate (): String
+ displayDetails ()
DisabledVehicle
- DisabledVehicle ()
<<AddVehicleGUI>>
CarParkingManagementSystem
+ CarParkingSpaceSLL ()
+ VehicleQueue ()
+ DisabledVehiclePriorityQueue ()
+ CarParkingHistoryStack ()
+ parkVehicle ()
+ removeVehicle ()
ElectricCar
+ ElectricCar ()
<<RemoveVehicleGUI>>
Car
- Car ()
CarParkingSpaceSLL
+ isTaken: boolean
+ vehicle: Vehicle
+ carSpaceID: String
+ space: CarParkingSpace
+ CarParkingSpaceSLL ()
+ setIsTaken (isTaken: boolean)
+ setVehicle (vehicle: Object)
+ setSpace(CarParkingSpace: Object)
+ setCarSpaceID (carSpaceID: String)
+ getIsTaken (): boolean
+ getSpace(): Object
+ getCarSpaceID(): String
+ getVehicle(): Object
+ findFreeSpace ()
+ addSpace ()
+ getVehicle (): Object
+ getCarSpaceID (): String
VehicleQueue
+ queue: ArrayList <Vehicle>
+ VehicleQueue ()
+ enqueue ()
+ dequeue ()
+ peek ()
+ isEmpty ()
+ displayQueue ()
<<ViewCarParkingSpacesGUI>>
CarParkingHistoryStack
+carEvent: String
+ historyStack: ArrayList <CarParkingHistory>
+ licensePlate: String
+ CarParkingHistoryStack ()
+ setHistoryStack (historyStack: ArrayList <CarParkingHistory>
+ setCarEvent (carEvent: String)
+ setLicensePlate (licensePlate: String)
+ getHistoryStack (): ArrayList <CarParkingHistory>
+ getLicensePlate (): String
+ getCarEvent (): String
+ push ()
+ pop ()
+ peek ()
+ isEmpty ()
+ displayRecord ()
+ displayHistoryStack ()
DisabledVehiclePriorityQueue
+ priorityQueue: ArrayList <Vehicle>
+ enqueue ()
+ dequeue ()
+ peek ()
+ isEmpty ()
+ displayPriorityQueue ()
<<DisabilityWaitingListGUI>>
<<CarParkingRecentHistoryGUI>>